文档简介

介绍 `rtos` 中 `gpadc` 的使用方法，为 `gpadc` 设备驱动开发提供参考。

目标读者

`gpadc` 驱动的开发/维护人员。

适用范围

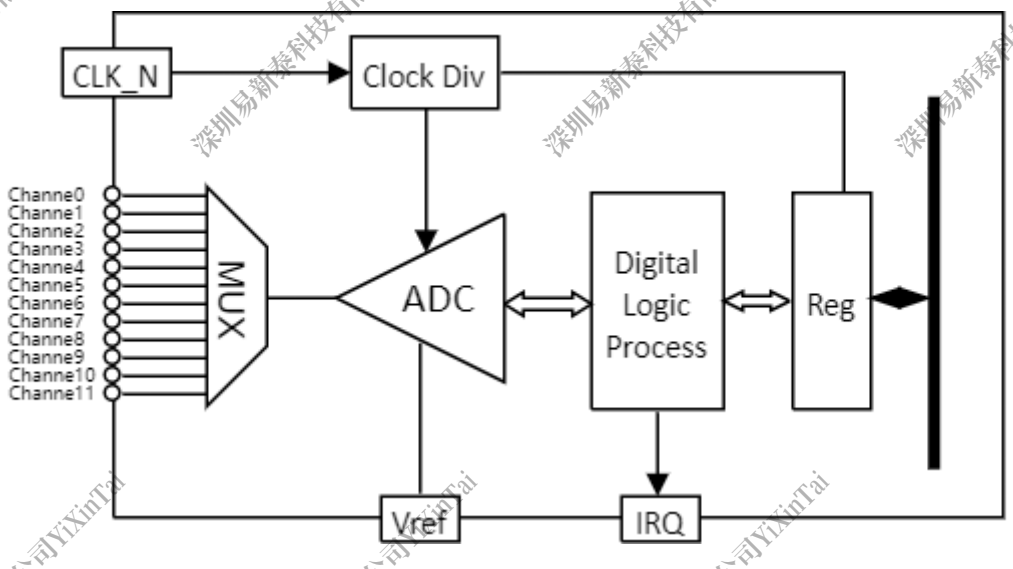
Table:适用产品列表

- MR527
- A523
- AI895
- MR153
- F135
- F102

相关术语介绍

术语	解释说明
sunxi	指Allwinner一系列soc硬件平台
GPADC	即General-Purpose Analog-to-Digital Converter，通用型模拟-数字转换器，其主要功能是将连续变化的模拟信号转换为离散的数字信号。常用于电池电压监测、密钥检测、过流检测、湿度传感器等场景。

模块功能介绍



GPADC是12-bit采集精度的模数转换模块，支持24路通道，模拟输入范围0 ~ 1.8 V，最高采样率1 MHz，并且支持数据比较，自校验功能，同时工作于可配置的四种工作模式：

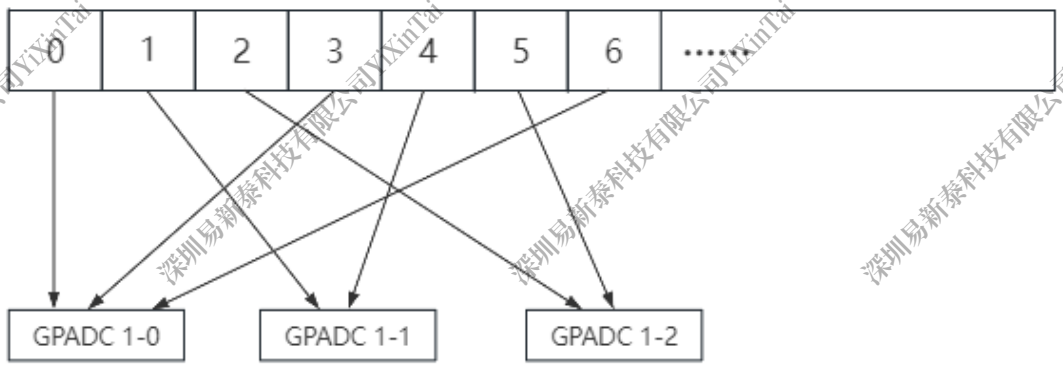
1.**Single模式**：单次采样模式。在以此模式使能GPADC后，在指定通道进行一次单次采样，**在完成一次采样后就停止采样**，并将结果保存在对应通道的数据寄存器GP_CH(x)_DAT(其中(x)为对应通道号)中。

（2）Single-cycle mode： 在指定的通道完成一个周期转换并将数据放在响应数据寄存器中；

（3）**Continue模式**：连续采样模式。在以此模式使能GPADC后，对指定通道进行连续采样(直到关闭ADC使能)，并将结果更新在对应通道的数据寄存器中，每次读取GP_CHx_DAT(其中(x)为对应通道号)寄存器都会读取到最新的值。

（4）**Burst模式**：连续采样模式。在**使能后会一直以设置好的采样率对指定通道连续采样**(直到关闭ADC使能)，并将结果存放入数据FIFO中，直到**满足一定数量后触发中断，并自动关闭。**

使用 Burst 需要设置触发数据量 (Trigger_level)，在数据量达到设置的触发数据量时，会触发中 断去调用用户注册的回调函数。同一端口下的不同通道共享同一个 FIFO，按通道顺序排列。以 MR527 为例，MR527 的每个 port 具有 64byte 深度的 FIFO，如果使能了 Port1 的 0/1/2 通道，那 么 Port1 的 FIFO 中的第一个数据为 channel0 的数据，第二个数据为 channel1 的数据，第三个数 据为 channel2 的数据，第四个数据为 channel0 的数据……以此类推。



::: note

GPADC仅在Burst模式下支持使用DMA搬运。

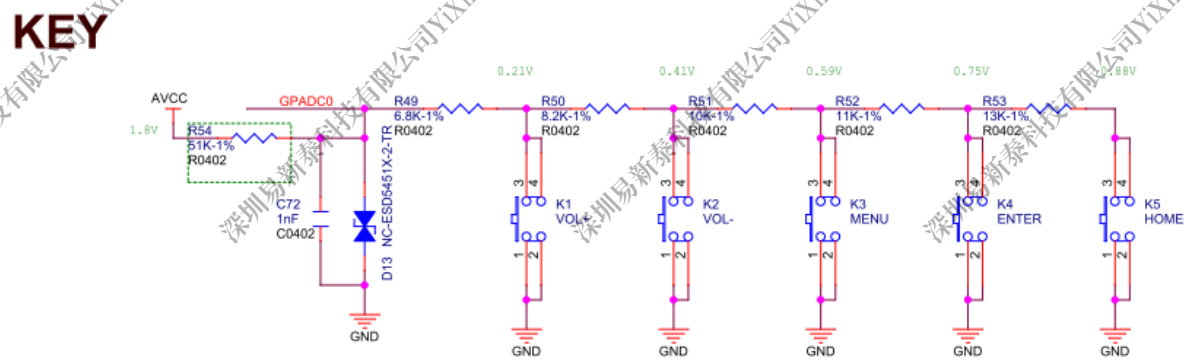
:::

PWM联动触发功能

平台	MR527	MR536	MR153
是否支持	否	是	是

在一些较新的芯片平台中支持GPADC与PWM的pulse模式相结合，在PWM设置了指定的Pulse个数并发送完成后，自动开启GPADC的采样使能位GP_EN。

如果有必要，也允许设置在收到触发条件后延迟一段时间再触发GPADC的采样使能，**这个延迟时间会被驱动转化为CLK的个数，这个CLK个数必须为 $$2^{n} \quad (n = 0, 1, 2, \ldots, 14)$$**（驱动中会自动选择最接近的延迟时间的满足要求的CLK数）。



部分`GPADC`接口用于KEY模块按键的读取，一般包括`VOL+、VOL-、HOME、MENU、ENTER`等等，GPADC0用于KEY的电路如上图。AVCC-AP为1.8 V的供电，不同的按键按下，GPADC0口的电压不同，CPU通过对这个电压的采样来确定具体是那一个按键按下。如上图，`VOL+、VOL-、MENU、ENTER、HOME/UBOOT`对应的电压分别为 0.21V、0.41V、0.59V、0.75V、0.88V。

模块配置介绍

kernel menuconfig 配置说明

```
.config - BitThunder Configuration
> Drivers Options > soc related device drivers > SUNXI GPADC Devices
                                                                  enabl
  There is no help available for this option.
  Symbol: DRIVERS_SUNXI_GPADC [=y]
  Type  : boolean
  Prompt: enable gpadc-ng driver
    Location:
      -> Drivers Options
        -> soc related device drivers
          -> SUNXI GPADC Devices
    Defined at drivers/rtos-hal/hal/source/gpadc-ng/Kconfig:3
```

源码结构介绍

GPADC驱动的相关源代码位于内核`drivers/hal/source/gpadc-ng`目录下：

```
source/gpadc-ng/
├── common_gpadc.h
├── hal_gpadc_ng.c
├── Kconfig
├── Makefile
├── platform
|   └── gpadc_sun55iw3.h
|   └── gpadc_sun55iw6.h
|   └── gpadc_sun8iw22.h
└
```

模块接口说明

GPADC初始化接口

- 函数原型： int hal_gpadc_init(int port);

- 功能描述：初始化指定`gpadc`模块，使能相应资源
- 参数说明：port：指定`gpadc id`
- 返回值：0：成功；非0：失败

GPADC通道配置接口

- 函数原型：hal_gpadc_status_t hal_gpadc_channel_init(int port, hal_gpadc_channel_t channel, bool keyadc)
- 功能描述：初始化指定`gpadc`的通道
- 参数说明：port：指定需要初始化的`gpadc`端口；channel：指定初始化的`channel`；keyadc：表明当前是否需要`gpadc`做按键，是传参为1，若不是，请传0；
- 返回值：0：成功；非0：失败

GPADC通道取消配置接口

- 原型：hal_gpadc_status_t hal_gpadc_channel_exit(int port, hal_gpadc_channel_t channel)
- 功能：取消GPADC某个通道配置
- 参数：`port`：指定`gpadc_id` `channel`：通道号
- 返回值：0: 成功；非0：失败

GPADC注册回调接口

- 原型：hal_gpadc_status_t hal_gpadc_register_callback(int port, hal_gpadc_channel_t channel, gpadc_callback_t user_callback)
- 功能：向应用层提供注册回调接口的功能
- 参数：`port`：指定`gpadc_id` `channel`：通道号 `user_callback`：应用层回调接口
- 返回值：0:成功；非0：失败

GPADC读取ADC接口

- 函数原型： uint32_t gpadc_read_channel_data(int port, hal_gpadc_channel_t channel)
- 功能描述：读取`adc`数值
- 参数说明：port：指定`gpadc id` channel：指定`adc`通道
- 返回值： 0：成功；非0：失败

GPADC去初始化接口

- 原型：hal_gpadc_status_t hal_gpadc_deinit(int port)
- 功能：GPADC模块去初始化
- 参数：指定`deinit`哪个端口
- 返回值：0:成功；非0：失败

深圳易新泰科技有限公司YiXinTai

接口使用

代码应用流程

步骤1：初始化GPADC端口，对硬件资源和软件内部参数进行初始。

```
hal_gpadc_init(port);
```

步骤2：配置GPADC的相关参数

```
    gpadc_config->channel_used = GP_CH0_SELECT | GP_CH1_SELECT | GP_CH2_SELECT;
    /* 设置gpadc的工作模式*/
    gpadc_config->mode = GP_BURST_MODE;
    gpadc_config->data_num_per_ch = 1200;
    gpadc_config->use_dma = true;
    gpadc_config->sample_rate = 2000000;

    hal_gpadc_config(port, gpadc_config);
```

步骤3：注册GPADC的callback

```
hal_gpadc_register_callback(port, channel, callback, para);
```

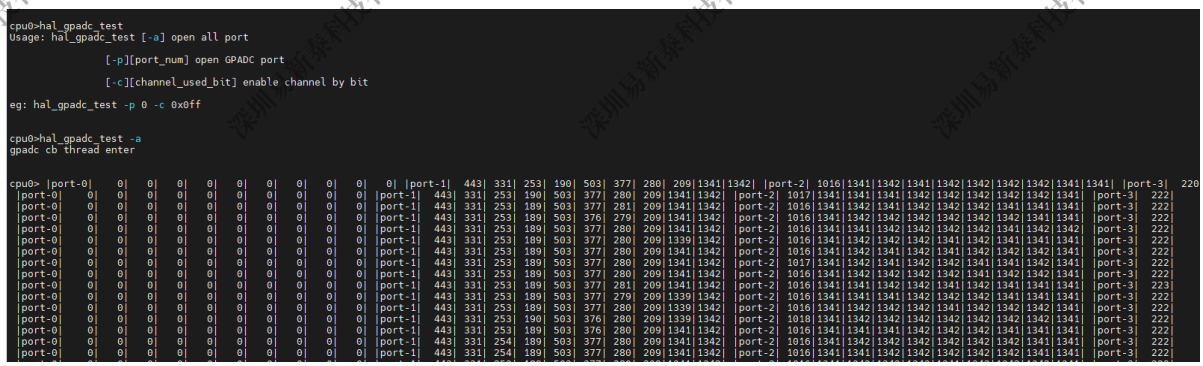
步骤4：使能GPADC

```
hal_gpadc_start(port);
```

测试用例介绍

测试用例说明

在选上对应的驱动、应用demo后便可使用GPADC来读取电压，在RV端开启并打印所有GPADC port的通道数据，使用命令**hal_gpadc_test -a**。



也可以单独使能一个端口 **hal_gpadc_test -p 1 -c 0x100**。

命令参数-p 1选择GPADC控制器1，第二个参数-c 0x100为选择GPADC对应的通道，0x100代表bit8，即GPADC1通道8；板端是1.34V接到GPADC1_8，可读取到data为1341mV，与输入电压一致。

测试代码解析

应用demo的代码路径在rtos/lichee/rtos-hal/hal/test/gpadc/test_gpadc.c

上节demo输入的两个参数分别为port（选择GPADC控制器）和channel（GPADC控制对应的通道）

使用函数步骤流程介绍如下（相关参数定义省略，详细可见代码）：

```
int sunxi_gpadc_all_test(int argc, char **argv)
{
    ……
    gpadc_config = hal_malloc(sizeof(hal_gpadc_config_t));
    memset(gpadc_config, 0 ,sizeof(hal_gpadc_config_t));
    gpadc_config->mode = GP_CONTINUOUS_MODE;
    gpadc_config->sample_rate = 1000000;
    gpadc_config->channel_used = (GP_CH0_SELECT | GP_CH1_SELECT | GP_CH2_SELECT
            | GP_CH3_SELECT | GP_CH4_SELECT | GP_CH5_SELECT | GP_CH6_SELECT
            |GP_CH7_SELECT |GP_CH8_SELECT |GP_CH9_SELECT;

    for (i = 0; i < GPADC_PORT; i++) {
        /* 初始化gpadc端口 */
        hal_gpadc_init(i);
        /* 将写好参数的结构体配置到gpadc-port */
        hal_gpadc_config(i, gpadc_config);
        /* 使能gpadc */
        hal_gpadc_start(i);
    }
    ……
    /* 创建线程，在线程中读取数据，详情请看代码文件 */
    ……
    return 0;
}
FINSH_FUNCTION_EXPORT_CMD(sunxi_gpadc_all_test, hal_gpadc_test, gpadc hal all
channel tests)

/* 以下线程用于循环打印gpadc端口的值 */
void gpadc_test_thread(void *param)
{
    ……
    while(1) {
        for (; i < GPADC_PORT; i++) {
            printf(" |port-%d| ", i);
            /* 获取当前gpadc-port的配置参数(如果不知道的话) */
            hal_gpadc_get_config(i, gpadc_config);

            for (j = 0; j < GPADC_CHANNEL_MAX[i]; j++) {
                    if ((gpadc_config->channel_used >> j) & 0x01) {
                        /* 读取数据并打印 */
                        vol_data[j] = gpadc_read_channel_data(i,j);
                        printf("%4d|", vol_data[j]);
                    }
            }
        }
        printf("\n");
        hal_msleep(10);
    }
```

深圳易新泰科技有限公司YiXinTai

```
    /* 反初始化gpadc，释放资源 */
    for (i = 0; i < GPADC_PORT; i++)
        hal_gpadc_deinit(i);
    hal_thread_stop(NULL);
}
```

Burst模式

测试用例说明

在选上对应的驱动、应用demo后，在RV端使用GPADC 0_8读取对应的电压输入，在小机端可使用如下命令：

```
hal_gpadc_ng_burst 1 0x03 40 1 1000000 10000
```

得到log如下：

```
命令原型：
hal_gpadc_ng_burst [port] [channel_bit] [data_len] [use_dma] [sample_rate]
[period_us]

RTOS串口控制台：
cpu0>hal_gpadc_ng_burst 1 0x03 10 1 1000000
Run gpadc test with burst mode

Linux串口控制台：
root@TinaLinux:/# cat /sys/kernel/debug/remoteproc/remoteproc0/aw_trace_log?[J
Run gpadc test with burst mode
[dma] port 1 channel 0 :
[0]1011
[1]1011
[2]1011
[3]1011
[4]1011
[5]1011
[6]1011
[7]1011
[8]1011
[9]1011

[dma] port 1 channel 1 :
[0]2040
[1]2039
[2]2039
[3]2039
[4]2040
[5]2040
[6]2039
[7]2039
[8]2039
[9]2039
```

输入参数作用如下：

参数名	功能介绍
port	GPADC控制器ID（MR536有4个GPADC控制器，MR527有2个）
channel_used	GPADC通道（每个GPADC对应若干个通道，详细参考用户手册）
data_num_per_ch	每个通道采集的数据长度，即每个通道采样多少个数据之后再一起触发port中断。
use_dma	本次采样是否使用dma
sample_rate	本次采样的采样率

上述命令中，第一个参数0代表选择了GPADC控制器port 0，第二个参数0x03是位掩码bit0和bit1，代表选择了port1的channel 0和channel 1，第三个参数20代表着采集20个数据触发一次中断，在中断中读取一次数据。

在使用burst模式时，同一个port口下的所有channel都会被设置为burst模式，**共享FIFO和FIFO中断。**

可以使用以下接口去读取指定port-channel的数据：

```
hal_gpadc_fifo_read_channel_data(int port, hal_gpadc_channel_t channel, uint16_t *data_buf)
```

::: note
由于共享FIFO中断，同一port口不同channel的callback由驱动分发。**此处设置的data_len是单个channel的data_len**，即如果开启了2个通道，data_len设置为10，则当每个通道采样到10个数据，共计20个数据时则触发Port的中断，按顺序调用注册channel的callback。
:::

测试代码解析

使用函数步骤流程介绍如下（相关参数定义省略，详细可见代码）：

```
int sunxi_gpadc_burst_test(int argc, char **argv)
{
    int ret = -1;
    struct gpadc_test_para_t gpadc_para;

    //初始化默认参数
    /* 配置端口的采样率 */
    gpadc_para.cfg.sample_rate = DEFAULT_SR;
    /* 配置Port模式：初始化为Burst模式 */
    gpadc_para.cfg.mode = GP_BURST_MODE;
    /* 配置是否使用DMA */
    gpadc_para.cfg.use_dma = true;
    /* 配置使能的通道 */
    gpadc_para.cfg.channel_used = GP_CH0_SELECT | GP_CH1_SELECT;

    /* 初始化GPADC的硬件端口 */
    ret = hal_gpadc_init(gpadc_para.port);

    /* 将写好的参数配置进port */
    hal_gpadc_config(gpadc_para.port, &gpadc_para.cfg);
```

```
    /* 申请读取数据的buffer */
    gpadc_para.data_buf = hal_malloc(gpadc_para.cfg.data_num_per_ch
                            * GPADC_CHANNEL_MAX[gpadc_para.port] *
sizeof(uint16_t));

    /* 注册中断的callback */
    hal_gpadc_register_callback(gpadc_para.port, gpadc_para.channel,
              sunxi_gpadc_irq_callback_for_channel, &gpadc_para);

    /*使能gpadc，开始采样*/
    hal_gpadc_start(gpadc_para.port);

    return 0;
}
FINSH_FUNCTION_EXPORT_CMD(sunxi_gpadc_burst_test, hal_gpadc_ng_burst, gpadc hal
APIs tests)

/* 采样结束触发中断时调用的回调函数 */
static int sunxi_gpadc_irq_callback_for_channel(void *param)
{
    ……
    /* 如果传参没有传入channel_used等配置参数，可以使用这个函数去获取
    // hal_gpadc_config_t cfg;
    // hal_gpadc_get_config(gpadc_para->port, &cfg);
     */

    /* 判断该通道是否使能 */
    for (i = 0; i < GPADC_CHANNEL_MAX[gpadc_para->port]; i++) {
        if (!(gpadc_para->cfg.channel_used & (0x01 << i)))
            continue;
        /* 读取数据 */
        len = hal_gpadc_fifo_read_channel_data(gpadc_para->port,
              i, gpadc_para->data_buf);

        /* 处理数据：此处将数据打印出来 */
        amp_printf("%s port %d channel %d :\n",
                gpadc_para->cfg.use_dma ? "[dma]" : "[fifo]", gpadc_para->port,
i);
        for (j = 0; j < len; j++)
            amp_printf("[%d]%u \n", j, *(gpadc_para->data_buf + j));
        amp_printf("\n");
    }
    ……
    return 0;
}
```

PWM联动触发

测试用例说明

测试用例代码路径如下：

```
rtos/lichee/rtos-hal/hal/test/pwm_gpadc/test_pwm_gpadc.c
```

除了GPADC驱动及demo需要使能外，还需要打开PWM的驱动。

深圳易新泰科技有限公司YiXinTai

```
BitThunder Configuration
└─> Drivers Options
    └─>soc related device drivers
        └─>SUNXI GPADC Devices
        └─>SUNXI PWM Devices
            └─> [*] enable pwm-ng driver
```

在选上对应的驱动、应用demo后编译烧录即可。

该测试用例使用PWM1-0触发GPADC2-0，在配置完成PWM的发送指定的9个周期波形后，延时170us自动使能GPADC2-0的burst模式进行采集，采集完毕返回一个callback。

在小机端输入如下命令：

```
cpu>hal_pwm_trigger_gpadc
```

得到log如下：

```
RV端：
uart>hal_pwm_trigger_gpadc
gpadc cb thread enter

Linux端：
cat /sys/kernel/debug/remoteproc/remoteproc0/aw_trace_log
gpadc cb thread enter

[dma] port 2 channel 0 :
[0]186
[1]184
[2]182
[3]180
[4]178
[5]175
[6]173
[7]172
[8]169
[9]167
[10]165
[11]164
……
……
[1190]3920
[1191]3923
[1192]3927
[1193]3928
[1194]3930
[1195]3932
[1196]3935
[1197]3937
[1198]3939
[1199]3941
```

深圳易新泰科技有限公司YiXinTai

测试代码解析

使用函数步骤流程介绍如下（相关参数定义省略，详细可见代码）：

```
int sunxi_triger_gpadc_test(int argc, char **argv)
{
    ……

    /* 配置PWM的相关参数pwm_cfg */
    pwm_cfg = hal_malloc(sizeof(struct pwm_config));
    ……
    hal_pwm_init(pwm_port); /*初始化PWM端口*/

    /* 配置GPADC的相关参数gpadc_para */
    gpadc_para->port = 2;
    gpadc_para->channel = 0;
    gpadc_para->delay_ns = 168000;       //在PWM发送完毕后间隔168 μs触发GPADC采样
    gpadc_para->cfg.channel_used = GP_CH0_SELECT;
    gpadc_para->cfg.mode = GP_BURST_MODE;
    gpadc_para->cfg.data_num_per_ch = 1200;
    gpadc_para->cfg.use_dma = true;
    gpadc_para->cfg.sample_rate = 2000000;

    ……
    /* 初始化GPADC */
    ret = hal_gpadc_init(gpadc_para->port);
    if (ret) {
        hal_log_err("[gpadc%d] init failed!\n" , gpadc_para->port);
        return -1;
    }
    /* 初始化相关配置 */
    hal_gpadc_config(gpadc_para->port, &gpadc_para->cfg);

    /* 注册一个dma的callback函数，在完成数据接收时触发 */
    hal_gpadc_register_callback(gpadc_para->port, gpadc_para->channel,
sunxi_gpadc_callback, gpadc_para);
    /* 设置GPADC为外部信号（PWM）触发模式，并设置触发延时 */
    hal_gpadc_external_trigger_set(gpadc_para->port, gpadc_para->delay_ns);

    hal_gpadc_start(gpadc_para->port);
    /* 设置PWM的外部触发 */
    hal_pwm_trigger_gpadc_enable(pwm_port, pwm_channel);
    /* 配置PWM并使能输出 */
    hal_pwm_control(pwm_port, pwm_channel, pwm_cfg);

    hal_msleep(3000);

    /* 释放相关资源 */
    hal_pwm_release(pwm_port, pwm_channel, pwm_cfg);
    hal_pwm_deinit(pwm_port);
    hal_gpadc_deinit(gpadc_para->port);
    hal_free(gpadc_para->data_buf);
    hal_free(pwm_cfg);

    return 0;
```

深圳易新泰科技有限公司YiXinTai

```
}

static int sunxi_gpadc_callback(void *param)
{
    struct gpadc_test_para_t *gpadc_para = (struct gpadc_test_para_t*) param;
    /* 回调函数中读取数据 */
    len = hal_gpadc_fifo_read_channel_data(gpadc_para->port, gpadc_para->channel,
gpadc_para->data_buf);

……
    return 0;
}
```

FAQ

Q1.当怀疑GPADC读的电压不准确/发生数据丢失时

现象描述： GPADC读取的电压值与实际输入电压存在较大偏差，或者采样过程中出现数据丢失、重复等问题。

可能原因：

- 输入电压源不稳定或存在噪声干扰
- Burst模式下FIFO溢出或数据处理不及时
- 寄存器配置不当（时钟源、采样率、采样时间等参数设置不合理）
- 硬件连接问题或电路设计缺陷

解决方法：
①使用万用表/示波器测量输入电压源，并且使用Max/Min功能记录峰-峰值，确认是否由于输入电压抖动引起的误差。

②如果使用的是Burst模式，可以在驱动中打开CONFIG_DRIVERS_GPADC_DEBUG宏，使用 **gpadc_fifo_debug [port]** 命令，该命令可以打印必要的参数，包括寄存器、软件buffer中的数据等，确认是丢失数据还是多出数据。

③将指定Port下功能的相关寄存器dump出来（方法参考《读写寄存器》），检查时钟源、模式、采样率和采样时间是否与设置的一致，且是否在合理范围内。具体配置关系可以查看对应《User Manual》GPADC章节下对应的《Clock and Timing Requirement》小节，通常情况下，采样时间TACQ的值不应小于0x02。

Q2.当GPADC的采样率达不到预设的值时/调用回调函数的时间过长

现象描述： GPADC实际采样率低于配置的采样率，或者采样完成后回调函数被调用的延迟时间过长，影响实时性要求。

可能原因：

- 系统时钟配置不当或时钟源频率不足
- 采样时间参数设置过长，导致单次采样耗时增加
- 中断优先级设置不当，其他高优先级中断频繁占用CPU
- 回调函数处理逻辑复杂，执行时间过长
- DMA传输效率低下或FIFO处理不及时

深圳易新泰科技有限公司YiXinTai

解决方法：
①可以接入信号发生器，抓取一个周期内规律变化的波形。将采样数据和实际波形对比，定位是采样时间过长还是采样完成但callback没有被立刻调到。

如果是前者，参考问题回答A1的点②和点③；如果是后者，可以使用**list_irq** 命令排查是否有其他中断同时触发，再排查是否其他中断的处理时间过长。

日志分析

GPADC驱动提供了多种日志级别，帮助开发者分析和调试问题：

日志级别说明

GPADC_ERR - 错误日志

用于记录严重的错误情况，如参数错误、硬件初始化失败等：

```
GPADC_ERR("port%d invalid!\n", port);
GPADC_ERR("[%d]port%u is error\n", __LINE__, (unsigned int)port);
```

日志特征：包含错误位置信息(**LINE**)和详细的错误描述。

GPADC_INFO - 信息日志

用于记录重要的操作状态和关键信息：

```
GPADC_INFO("gpadc callback, param: %p\n", param);
GPADC_INFO("gpadc set sample rate");
```

日志特征：提供操作流程的关键节点信息。

GPADC_DEBUG - 调试日志

在CONFIG_DRIVERS_GPADC_DEBUG宏定义时启用，提供详细的调试信息：

```
#ifdef CONFIG_DRIVERS_GPADC_DEBUG
int hal_gpadc_fifo_debug(int argc, const char **argv)
{
    // 输出详细状态信息
}
#endif
```

日志特征：包含寄存器状态、数据缓冲区内容等详细信息。

日志查看方法

RTOS端查看

通过串口直接查看日志输出，实时观察GPADC运行状态。

深圳易新泰科技有限公司YiXinTai

调试方法

调试步骤

基本功能调试

1. 确认硬件初始化成功
2. 验证通道配置正确
3. 检查时钟和复位配置

采样调试

1. 使用单次模式测试基本采样功能
2. 切换到Burst模式测试连续采样
3. 验证DMA传输是否正常

性能调试

1. 调整采样率观察系统稳定性
2. 优化FIFO阈值设置
3. 分析中断处理效率

深圳易新泰科技有限公司YiXinTai